

Supply chain scheduling: Sequence coordination

Alessandro Agnetis^a, Nicholas G. Hall^b, Dario Pacciarelli^c

^a*Dipartimento di Ingegneria dell'Informazione, Università di Siena, via Roma 56, 53100 Siena, Italy*

^b*Department of Management Sciences, The Ohio State University, 2100 Neil Avenue, Columbus, OH 43210-1144, USA*

^c*Dipartimento di Informatica e Automazione, Università Roma Tre, via della Vasca Navale 79, 00146 Roma, Italy*

Received 12 September 2002; received in revised form 4 July 2004; accepted 21 April 2005

Available online 18 April 2006

Abstract

A critical issue in supply chain management is coordinating the decisions made by decision makers at different stages, for example a supplier and one or several manufacturers. We model this issue by assuming that both the supplier and each manufacturer have an ideal schedule, determined by their own costs and constraints. An interchange cost is incurred by the supplier or a manufacturer whenever the relative order of two jobs in its actual schedule is different from that in its ideal schedule. An intermediate storage buffer is available to resequence the jobs between the two stages. We consider the problems of finding an optimal supplier's schedule, an optimal manufacturer's schedule, and optimal schedules for both. The objective functions we consider are the minimization of total interchange cost, and of total interchange plus buffer storage cost. We describe efficient algorithms for all the supplier's and manufacturers' problems, as well as for a special case of the joint scheduling problem. The running time of these algorithms is polynomial in both the number of jobs and the number of manufacturers. Finally, we identify conditions under which cooperation between the supplier and a manufacturer reduces their total cost.

© 2006 Elsevier B.V. All rights reserved.

Keywords: Manufacturing; Supply chain scheduling; Cooperation; Polynomial time algorithm

1. Introduction

A *supply chain* represents all the stages at which value is added to a manufacturing product, including the supply of raw materials and intermediate components, finished goods manufacture, packaging, transportation, warehousing and logistics. A central issue in supply chain management is *coordination* between decisions made at different stages of the supply chain, for example between a supplier and a manufacturer. Although the benefits of coordination may be substantial, the mathematical modelling of coordinated decision making in supply chains is still a largely undeveloped area of research. Banker and Khosla [3] identify a need for research on supply chain management practices that provide a competitive advantage. Karmarkar [14] recommends an integrated approach to operations management and marketing decisions.

In an extensive review paper, Thomas and Griffin [23] identify several streams of supply chain management research, such as coordinated planning in inventory–distribution systems [18], coordination in production–distribution systems [5], and buyer–vendor coordination [2]. They also point out the need for research addressing *deterministic* and *operational* supply chain issues rather than stochastic and strategic ones. This paper addresses that need.

E-mail addresses: agnetis@dii.unisi.it (A. Agnetis), hall.33@osu.edu (N.G. Hall), pacciarelli@dia.uniroma3.it (D. Pacciarelli).

Sarmiento and Nagi [22] survey the literature of integrated production and distribution models, and point out that the recent trend towards reduced inventory levels creates a need for greater coordination between decisions at different stages of a supply chain. Several authors consider production systems with multiple decision makers. Ow et al. [21] discuss a multi-agent, or distributed, scheduling system, but do not develop scheduling algorithms. Weng [24] develops models to find the optimal sales price and production/order quantity for each stage of a two-stage supply chain, and examines the benefit from cooperation. Moses and Seshadri [17] develop a model that finds an optimal review period and a stocking policy for a two-stage supply chain.

This paper considers a number of operational issues that are important to the scheduling of supply chains organized to achieve just-in-time (JIT) goals [20,12]. The study of such issues is called *supply chain scheduling* [10]. At the operational level, decision makers at different stages of the chain need to consider various factors such as their immediate customers' due dates, and production deadlines, changeover costs and times. As a result, each stage defines its own ideal schedule that specifies how orders should be processed at that stage. For example, an assembly facility which has to ship jobs to different customers may wish to process the materials in the same sequence as the due dates. On the other hand, according to JIT concepts, scheduling decisions at an upstream stage must also comply with the actual time at which the supplier will dispatch the raw materials and with technological requirements that may make certain schedules infeasible. Indeed, Nellemann and Smith [19] identify the punctual delivery of materials at different stages to be among the most important elements of a successful JIT system. Thus, the schedule that is used at each stage depends on the requirements at the other stages.

The importance of coordination is emphasized by the results of a survey [11], in which many suppliers to US and Japanese automobile manufacturers agreed that "JIT only transfers inventory responsibility from customers to suppliers". As observed by Erengüç et al. [8], this is due to the fact that JIT deliveries are not matched with JIT production, so that manufacturers stockpile inventory in order to meet their customers' demands. Lee and Chen [15] study a variety of mathematical models related to the integration of transportation and scheduling decisions, but without addressing supply chain conflict issues.

Supply chain scheduling coordination issues arise in a number of specific practical situations. For example, Blumenfeld et al. [4] consider a manufacturing model where one producer has several customers, each cyclically receiving a given product type. The production schedule is based on setup and inventory costs, however the distribution schedule depends on freight and load inventory costs. Their objective is the minimization of overall inventory costs, and they analyze the trade-off between the benefits of coordination and increased management complexity. Similarly, Hurter and Van Buer [13] study a problem of coordination between the printing (i.e., production) and distribution departments in a newspaper company. Here the printing department prefers to manufacture jobs according to an ideal schedule that minimizes overall production time, whereas the distribution department prefers that products which will be shipped over longer distances are produced first. The authors study the tradeoff between these conflicting preferences. A similar environment is described by Agnetis et al. [1]. Here, two consecutive departments in the production process of a furniture manufacturer rank the jobs on the basis of their color and size respectively, but a common job sequence that trades off the two departments' objectives is needed.

There are a few papers which address supply chain scheduling coordination issues at the operational level. Hall and Potts [10] study the benefits of cooperative decision making in a supply chain where a supplier makes deliveries to several manufacturers, who in turn supply several customers. They develop models which minimize the total of scheduling and batch delivery costs. Chen and Hall [6] analyze a variety of models that arise when several suppliers deliver their products to the same assembly facility. They demonstrate that the benefits of coordinated decision making can be substantial. Dawande et al. [7] provide a new analysis of the newspaper distribution problem, and also consider a closely related mixed model assembly problem.

In this paper, we consider two consecutive stages of a supply chain, consisting of one supplier and several manufacturers, respectively. The costs and requirements specific to each stage (for example, changeover times, resource availability and deadlines) define an ideal schedule in which the jobs should be processed. This ideal schedule minimizes overall costs subject to resource constraints at that stage. Then the supply chain scheduling coordination problem is to find a schedule for each stage that considers both stages' ideal schedules.

One possible situation we consider occurs when one of the two stages imposes its own ideal schedule on the other. Then the other stage has to optimize its own schedule, subject to this given schedule and a limited resequencing capability in a storage buffer. We are interested in minimizing the distance between the actual schedules at the various stages of the chain and their respective ideal schedules. The distance between the actual and ideal schedules at any

stage of the supply chain is measured by the minimum number of adjacent pairwise interchanges that are necessary to transform one schedule into the other. The pairwise interchange cost is a standard measure of the distance between sequences. It is used, for example, in genome sequencing [16].

More specifically, we think about an ideal sequence as being defined by changeover times, such as might occur in various process industries including paint, chemicals and fertilizer. Suppose that, in these applications, the ideal and least costly sequence $(1, 2, 3)$ is from light to dark, or from mild to toxic. In this case, the sequence $\sigma = (3, 1, 2)$ which has two interchanges is worse than the sequence $\sigma' = (1, 3, 2)$ which has only one interchange. This is because the “increasing” changeovers $(1, 2)$ in σ and $(1, 3)$ in σ' have zero or minimal cost, whereas the cost of the “decreasing” changeover $(3, 1)$ in σ is likely to be greater than that for $(3, 2)$ in σ' due to greater dissimilarity between the products.

This situation is studied from the point of view of the manufacturer, and then from that of the supplier. Both these problems are considered under two different objectives. In the first objective, there is an interchange cost but no storage cost. In the second objective, the addition of storage cost discourages the use of the buffer. A second situation we consider occurs when both stages compromise in order to reach a satisfactory overall schedule. That is, they both schedule the jobs in a less than ideal way, in order to achieve a fair redistribution of the overall costs. We study this problem under the objective of minimizing the interchange cost, and show by example that our results do not hold in the presence of buffer costs.

This paper is organized as follows. In Section 3, we describe our notation and scheme for problem classification, and we provide a brief overview of our results. In Section 3, we analyze several problems where one of the manufacturers has to comply with the supplier’s ideal schedule. In Section 4, the roles are reversed and the supplier has to meet several manufacturers’ ideal schedules. Section 5 considers the joint supplier-manufacturer decision problem that minimizes the total system cost, and identifies conditions for profitable cooperation between the supplier and the manufacturer. Some incentives and practical mechanisms for cooperation are also discussed. Section 6 contains some conclusions and suggestions for future research.

2. Notation and classification

Here we describe our notation and assumptions, and provide a brief overview of our results. Let $N = \{1, \dots, n\}$ denote a set of jobs to be processed. Each job is processed first by the supplier and then by the manufacturer who ordered it. A single supplier supplies G manufacturers. The time horizon is divided into T time slots. Processing of a job takes one time slot, for both the supplier and a manufacturer. We consider a balanced situation where each manufacturer can process at most one job in each time slot, whereas the supplier can process at most G jobs in each time slot. It is possible that two or more jobs processed by the supplier in a given time slot may have been ordered by the same manufacturer. A job which is processed by the supplier in time slot t can be processed by a manufacturer either in time slot $t + 1$, or later. In the latter case, the job must wait in an intermediate buffer of given capacity $b < n$. We assume that control of the buffer is exercised by the specified decision maker(s) in each problem considered. The set of available jobs in the buffer during time slot t is denoted by $\beta(t)$, where $|\beta(t)| \leq b$. Fig. 1 illustrates the arborescence structure of the two stages.

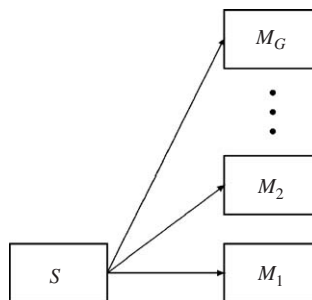


Fig. 1. Structure of the two stage supply chain.

In accordance with the discussion in Section 1, both the supplier and each manufacturer have an ideal schedule in which they would like to process the jobs. We denote by S , S^I , and S^* a feasible schedule of jobs for the supplier, an ideal schedule for the supplier, and an optimal schedule that minimizes the supplier's cost while meeting the manufacturer's requirements, respectively. A feasible schedule is one that does not exceed the available buffer capacity, b , at any time. Also, we denote by $S(t)$ (respectively, $S^I(t)$, $S^*(t)$) the set of jobs processed by the supplier during time slot t in schedule S (resp., S^I , S^*). We let $\sigma(j)$ (respectively, $\sigma^I(j)$, $\sigma^*(j)$) denote the time slot in which job j is processed by the supplier in schedule S (resp., S^I , S^*). Analogously for the manufacturers, we let M_r , M_r^I , and M_r^* denote a feasible schedule for the r th manufacturer, an ideal schedule for the r th manufacturer, and an optimal schedule for the r th manufacturer, respectively. We use the notation M , M^I , and M^* when referring to all the G manufacturers as a whole. Hence, $M(t)$ (respectively, $M^I(t)$, $M^*(t)$) denotes the set of jobs processed by the G manufacturers during time slot t in schedule M (resp., M^I , M^*), while $M_r(t)$ (resp., $M_r^I(t)$, $M_r^*(t)$) has the same meaning for the r th manufacturer. We also denote by $\mu(j)$ (respectively, $\mu^I(j)$, $\mu^*(j)$) the time slot in which job j is processed by some manufacturer in schedule M (resp., M^I , M^*).

If a decision maker processes one or more jobs during a time slot, then that time slot is said to be *active*; otherwise, it is *idle*. Consider the supplier's ideal schedule, S^I . We let $i \prec^S j$ denote that job i precedes job j in S^I . For the ideal sequence of the r th manufacturer, we similarly use the notation $i \prec^{M_r} j$, dropping the subscript where the identity of the manufacturer is clear from the context. We say that an *interchange* occurs whenever job i is processed strictly before j , whereas in the ideal schedule j strictly precedes i . Without loss of generality, we let the cost of an interchange relative to an ideal schedule be 1. The cost of storing one job in a buffer for one time slot is denoted by w . This cost can vary between the decision makers. Depending upon which model is being studied, the decisions to be made include finding an optimal supplier's schedule S^* , an optimal schedule M_r^* for the r th manufacturer where $1 \leq r \leq G$, or an optimal combined schedule S^* , M_r^* . The decision maker(s) are either the supplier, or the r th manufacturer, or both jointly.

The standard classification scheme for scheduling problems [9] is $\psi_1|\psi_2|\psi_3$, where ψ_1 indicates the scheduling environment, ψ_2 describes the job characteristics or restrictive requirements, and ψ_3 defines the objective function to be minimized. We let $\psi_1 = S$ where the decision maker is a supplier, and $\psi_1 = M_r$ where the decision maker is manufacturer M_r . Where both these parties are joint decision makers, we let $\psi_1 = S, M_r$. Under ψ_2 , we use b to describe the buffer capacity. The objective functions that we consider under ψ_3 are:

C the total cost of interchanges, relative to the decision maker's ideal schedule;

$C + W$ the total cost of interchanges plus storage.

In the remainder of this paper, we first provide polynomial time algorithms for several problems. For problem $M_r|b|C$, we describe an algorithm with time complexity $O(nb \log b)$. For problem $M_r|b|C + W$, we describe two different algorithms with time complexities $O(n^{b+2}b)$ and $O(n^5b^4)$, respectively. For problem $S|b|C$, we describe an algorithm with time complexity $O(nb \log(G + b))$. For problem $S|b|C + W$, we describe an algorithm with time complexity $O(n^{b+1}b(G + b) \log(G + b))$. We also show that achieving cost savings by cooperation between a supplier and a manufacturer is possible in problem $S, M_r|b|C + W$, but not in problem $S, M_r|b|C$.

Before considering several models for the manufacturer's and supplier's problems, we show that the number of time slots that need to be considered to construct an optimal schedule is small. For the sake of simplicity, we consider the case $G = 1$, but the extension to general G is straightforward.

Lemma 1. *The number of time slots in which processing may occur in an optimal schedule is $O(nb)$.*

Proof. Consider a feasible schedule for the supplier and a manufacturer, and suppose that the supplier releases one job at time t . There are $|\beta(t)|$ jobs in the buffer at time t . Suppose that the supplier releases the next job at time t' , where $t' > t + |\beta(t)| + 1$. Further, suppose that the manufacturer processes a job in a time slot between $t + |\beta(t)| + 2$ and t' . In this case, at least one time slot between $t + 1$ and $t + |\beta(t)| + 1$ is idle for both the supplier and the manufacturer. Removing this time slot clearly does not affect schedule feasibility, nor does it alter interchange costs. Storage costs, if present, can decrease but not increase as a result. Thus, for each of the $O(n)$ jobs released by the supplier, we can restrict ourselves to considering at most $b + 1$ time slots in the schedule. $\square$

In view of Lemma 1, we can a priori discard irrelevant time slots and hence number the remaining time slots $1, \dots, T$, where $T = O(nb)$, in all the analysis which follows.

3. Manufacturer's problems

Here we consider problems in which the decision maker is the r th of the G manufacturers. In this section of the paper, we let S^I denote the ideal supplier's schedule *relative* to this manufacturer, i.e., the ideal schedule of the jobs which are ordered by this manufacturer only. Hence, some job sets $S^I(t)$ may be empty for some time slots t . In the manufacturer's problem, the jobs are released by the supplier according to schedule S^I . The r th manufacturer also has its own ideal schedule, M_r^I . The manufacturer can resequence the jobs by storing them in a buffer when they arrive from the supplier, and then retrieving them in a sequence that is different from that in which they were received. However, the given capacity $b < n$ of the buffer limits the resequencing options of the manufacturer. Let z be the last time slot t during which jobs are released by the supplier, i.e. $z = \max\{t : |S(t)| > 0\}$.

3.1. Interchange costs

We first consider problem $M_r|b|C$, i.e., the minimization of interchange cost for the manufacturer. At each time slot, based on the currently available jobs in the buffer, the manufacturer decides whether or not to produce a job. We let $u_t = 1$ if time slot t is idle, and $u_t = 0$ if a job is scheduled at time t . Note that $u_1 = 1$, since the supplier releases no jobs before time slot 1. We refer to the vector $U = [u_1, \dots, u_T]$ as the *profile* of a schedule M_r . The profile of a schedule identifies which time slots are idle in the manufacturer's schedule, but does not specify the order in which the jobs are processed. We let $q_t = \sum_{i=1}^{t-1} |S^I(i)| - t$, and $Q_t = \max_{t \leq v \leq T} \{q_v\} - q_t = \max_{t \leq v \leq T} \{\sum_{i=t}^{v-1} |S^I(i)| - (v - t)\}$. Note that Q_t represents the minimum amount of buffer space that needs to be set aside for jobs arriving in the future, in order to ensure feasibility. In fact,

$$Q_t = \max_{v=t, \dots, T} \{q_v\} - q_t. \quad (1)$$

We note that this implies $Q_t \geq 0$.

Lemma 2. *A schedule for problem $M_r|b|C$ satisfies the buffer capacity constraint if and only if $q_t + \sum_{i=1}^t u_i \leq b$ for $t = 1, \dots, T$.*

Proof. Observe that from time slot 1 through time slot t the supplier supplies $\sum_{i=1}^{t-1} |S(i)|$ jobs, and only $t - \sum_{i=1}^t u_i$ of them have been processed by the manufacturer. Hence, the number of jobs in the buffer during time slot t is exactly $q_t + \sum_{i=1}^t u_i$. $\square$

Given a partial schedule from time slot 1 to time slot t , we let $\psi(t) = Q_t + q_t + \sum_{i=1}^t u_i$. Note that $\psi(t)$ is the total buffer requirement at time t , including both the space required to store the jobs currently in the buffer and the space that needs to be set aside for future jobs.

Lemma 3. *Given a partial schedule from 1 to t , if $\psi(t) > b$, then no feasible schedule exists from time $t + 1$ on.*

Proof. Given t , let $v^* \geq t$ be such that $Q_t = \sum_{i=t}^{v^*-1} |S^I(i)| - (v^* - t)$. Then, $Q_t + q_t = \sum_{i=1}^{v^*-1} |S^I(i)| - t - (v^* - t) = q_{v^*}$ by definition. Hence, if $\psi(t) > b$, and since $t \leq v^*$, we have $q_{v^*} + \sum_{i=1}^{v^*} u_i \geq q_{v^*} + \sum_{i=1}^t u_i = \psi(t) > b$. Then, from Lemma 2, it follows that no feasible schedule exists. $\square$

Lemma 4. $Q_{t-1} = \max\{0, Q_t + q_t - q_{t-1}\}$, for $t = 2, \dots, T$.

Proof. From (1) and $Q_{t-1} = \max_{t-1 \leq v \leq T} \{q_v\} - q_{t-1}$, we obtain $Q_{t-1} = \max\{q_{t-1}, Q_t + q_t\} - q_{t-1} = \max\{0, Q_t + q_t - q_{t-1}\}$. $\square$

Lemma 5. *In any feasible schedule, given a time slot t such that $u_t = 0$, we have $\psi(t) \leq \psi(t - 1)$.*

Proof. By definition, and since $u_t = 0$, we have $\psi(t) - \psi(t - 1) = (Q_t + q_t + \sum_{i=1}^t u_i) - (Q_{t-1} + q_{t-1} + \sum_{i=1}^{t-1} u_i) = (Q_t + q_t - q_{t-1}) - Q_{t-1} \leq 0$ from Lemma 4. $\square$

Consider a profile U such that $u_t = 0$ if and only if $q_t + Q_t + \sum_{i=1}^{t-1} u_i = b$ for $t = 1, \dots, z$. Intuitively, this means that the manufacturer delays the processing of the next job until the widest possible choice of jobs is available, which occurs when the buffer is full. A manufacturer's profile that satisfies this property is said to be *packed*. Observe that if a profile is packed, then the position of the idle time slots from 1 to z is uniquely determined by the supplier's ideal schedule S^I .

Lemma 6. *There exists an optimal schedule for problem $M_r|b|C$ that has a packed profile.*

Proof. First, observe that in a feasible schedule $u_t = 1$ implies $q_t + Q_t + \sum_{i=1}^{t-1} u_i = \psi(t) - u_t < b$. In fact, if $\psi(t) - u_t = b$ and $u_t = 1$, then from Lemma 3, a buffer overflow occurs at some time slot after t . Hence, to prove the lemma it is sufficient to show that, given any feasible schedule for problem $M_r|b|C$, there exists another feasible schedule with the same interchange cost and a packed profile. Consider any feasible manufacturer's schedule without a packed profile. Suppose there is a time slot t such that $u_t = 0$, $u_{t+1} = 1$ and $\psi(t) - u_t < b$. We can move the idle time earlier from $t + 1$ to t , hence delaying the processing of the job from time slot t to $t + 1$. This solution is still feasible, since in the new schedule $\psi(t)$ increases by 1 (and cannot therefore exceed b), and all the other $\psi(i)$ values, for $i = 1, \dots, T$, $i \neq t$ remain the same. Clearly, no new job interchange occurs when moving idle time slots earlier. By repeating this argument, we obtain a feasible schedule in which all the active time slots t preceding an idle time slot $t + 1$ satisfy the property $\psi(t) - u_t = b$. Alternatively, suppose that $u_t = 0$, $u_{t+1} = 0$ and $\psi(t + 1) - u_{t+1} = b$. Since $u_{t+1} = 0$, from Lemma 5, we have $\psi(t) \geq \psi(t + 1) = b$. Also, from Lemma 3, $\psi(t) \leq b$, and hence $\psi(t) = b$. By repeating this argument backwards in the schedule, we obtain $\psi(i) - u_i = b$ for all consecutive active time slots i between any two idle time slots. Thus, the profile is packed. $\square$

In a manufacturer's problem, given a set of jobs X , we say that $j \in X$ is the *leftmost* job in M_r^I if $j \prec^M k$ for all $k \in X$.

Lemma 7. *A schedule $\tilde{M}_r$ for problem $M_r|b|C$ has the minimum number of interchanges among all schedules having a given profile if and only if, whenever a job is scheduled in time slot $t + 1$, it is the leftmost in M_r^I among those in $\beta(t) \cup S(t)$.*

Proof. (Only if.) Consider jobs i and h such that $i \prec^M h$, where both belong to $\beta(t) \cup S(t)$. For purposes of contradiction, suppose that job h is scheduled before job i in a schedule $\tilde{M}_r$ having minimum interchange cost, and let M'_r be the schedule obtained by swapping i and h . Clearly, M'_r has the same profile as $\tilde{M}_r$. Since i and h are both in the buffer when job h is scheduled in $\tilde{M}_r$, it follows that M'_r is also feasible. Let $\mathcal{J}$ denote the set of jobs scheduled between h and i in $\tilde{M}_r$, where $\mathcal{J} = \emptyset$ is possible. The difference between the interchange costs in $\tilde{M}_r$ and M'_r is due only to i , h and the jobs in $\mathcal{J}$. In $\tilde{M}_r$, these interchange costs sum up to $|\{k : k \in \mathcal{J}, k \prec^M h\}| + |\{k : k \in \mathcal{J}, i \prec^M k\}| + 1$, where the last term is due to the interchange between i and h . In M'_r the interchange costs are $|\{k : k \in \mathcal{J}, k \prec^M i\}| + |\{k : k \in \mathcal{J}, h \prec^M k\}|$. Since $i \prec^M h$, we have $|\{k : k \in \mathcal{J}, k \prec^M i\}| \leq |\{k : k \in \mathcal{J}, k \prec^M h\}|$ and $|\{k : k \in \mathcal{J}, h \prec^M k\}| \leq |\{k : k \in \mathcal{J}, i \prec^M k\}|$. Thus, M'_r has a smaller interchange cost than $\tilde{M}_r$, a contradiction. Hence, at the time h is scheduled, it is the leftmost in M_r^I among the available jobs.

(If.) The result follows from the observation that, for a given profile, the schedule satisfying the necessary conditions is unique. $\square$

Lemmas 2 through 7 suggest the following algorithm.

Algorithm M-C.

Step 0: Given S^I , M_r^I , b . Let z be the last slot in which some job is released by the supplier.

Step 1: For $t = 1, \dots, z$, compute $q_t = \sum_{i=1}^{t-1} |S^I(i)| - t$, and $Q_t = \max_{t \leq v \leq T} \{\sum_{i=t}^{v-1} |S^I(i)| - (v - t)\}$.

Step 2: For $t = 1, \dots, T$:

If $Q_t + q_t + \sum_{i=1}^{t-1} u_i \geq b$ and there are no jobs available to the manufacturer, then no feasible schedule exists. Terminate.

If $Q_t + q_t + \sum_{i=1}^{t-1} u_i < b$, then schedule idle time and set $u_t = 1$.

Otherwise, schedule the available job which is leftmost in M_r^I , and set $u_t = 0$.

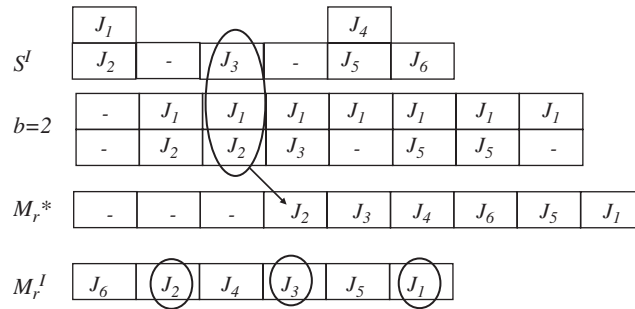


Fig. 2. Illustration of algorithm M-C.

Step 3: For $t = z + 1, \dots, T$:

Schedule the job in the buffer which is leftmost in M_r^I . Terminate.

Theorem 1. Algorithm M-C finds an optimal schedule for Problem $M_r|b|C$ in $O(nb \log b)$ time.

Proof. Algorithm M-C schedules idle time whenever $\psi(t) - u_t < b$. Hence, from Lemmas 2 through 5, the profile of the resulting schedule is packed, and therefore M-C finds a feasible solution if one exists. Finally, it follows from Lemmas 6 and 7 that the schedule found by Algorithm M-C has minimum cost.

We consider the time complexity of Algorithm M-C. From the definitions of q_t and Q_t , and from Lemma 4, it follows that Step 1 requires $O(T)$ time. Step 2 requires $O(T)$ repetitions of the choice of next job to schedule. The jobs in the buffer are stored in a list ordered by the M_r^I schedule. Each arriving job is inserted in this list, using binary search, in $O(\log b)$ time. The next (i.e., leftmost) job is taken from the front of the list. Step 2 requires $O(T)$ insertions in the list, each requiring $O(\log b)$ time. Therefore, the overall time complexity of Algorithm M-C is $O(T \log b)$, which from Lemma 1 equals $O(nb \log b)$. $\square$

Example 1. Consider the example in Fig. 2. There are $n = 6$ jobs and the buffer has capacity $b = 2$. The ideal supplier's and manufacturer's schedules are also shown in Fig. 2. The time slot in which the last job is released by the supplier is $z = 6$. At time 1, $u_1 = 1$. At time 2, $q_2 = |S^I(1)| - 2 = 2 - 2 = 0$ and $Q_2 = \max\{0, 0 - 1, 1 - 2, 1 - 3, 3 - 4, 4 - 5\} = 0$. Hence, $q_2 + Q_2 + u_1 = 1 < b = 2$, so again an idle time slot is scheduled ($u_2 = 1$). Similarly, $q_3 = -1$ and $Q_3 = 0$, $q_3 + Q_3 + u_1 + u_2 = -1 + 2 = 1 < 2$ and $u_3 = 1$. For $t = 4$, $q_4 = \sum_{i=1}^3 |S^I(i)| - 4 = -1$, $Q_4 = 0$ and hence $q_4 + Q_4 + u_1 + u_2 + u_3 = 2 = b$. Since there are available jobs (1, 2 and 3), the leftmost in M_r^I among them (i.e., job 2) is scheduled at $t = 4$, so that $u_4 = 0$. Continuing thus, $q_5 = -2$, $Q_5 = 1$ (obtained for $v = 6$), and therefore $q_5 + Q_5 + u_1 + u_2 + u_3 + u_4 = 2 = b$. Again there are available jobs, so job 3 is scheduled and $u_5 = 0$. At time $t = 6$, $q_6 = -1$, $Q_6 = 0$ and $q_6 + Q_6 + u_1 + u_2 + u_3 + u_4 + u_5 = 2 = b$, so job 4 is scheduled and $u_6 = 0$. After slot z , the buffer is emptied by repeatedly scheduling the leftmost job in M_r^I , which yields the remaining sequence (6, 5, 1).

3.2. Interchange and storage costs

We now consider the more general manufacturer's problem where the objective function also includes storage costs, $M_r|b|C + W$. Recall that the cost of holding one job in the buffer for one time slot is w . We describe two dynamic programming algorithms for problem $M_r|b|C + W$. The first, M-CW1, has running time that is polynomial in n , but exponential in the buffer capacity, b . The second, M-CW2, has running time that is polynomial in both n and b . However, neither algorithm is more efficient than the other for all problem instances. The following preliminary result simplifies the choice of which job to schedule next in both algorithms. The proof is similar to the "only if" part of Lemma 7 and is omitted.

Lemma 8. There exists an optimal schedule M_r^* for problem $M_r|b|C + W$, in which whenever a job is scheduled in time slot $t + 1$, it is the leftmost job in M_r^I among those in $\beta(t) \cup S(t)$.

In the description of Algorithm M-CW1 that follows, given a set B of jobs, we let v denote the leftmost job in the manufacturer's ideal schedule M_r^I among the jobs in $B \cup S(t)$. We also let $F(t, j)$ denote the number of jobs which are released by the supplier at or after time slot t and which precede job j in M_r^I .

Algorithm M-CW1.

Preprocessing

$F(t, j) = |\{i : i \in \bigcup_{h=t}^T S(h), i \prec^M j\}|$, for $t = 1, \dots, T$, $j = 1, \dots, n$.

Value Function

$f_t(B)$ = the minimum total (interchange plus storage) cost to solve the problem over the time slots from $t + 1$ through T , if $\beta(t) = B$.

Boundary Conditions

$f_T(\emptyset) = 0$.

$f_{t+1}(B \cup S(t) - \{j\}) = +\infty$, if $B \cup S(t) = \emptyset$, for $t = 0, \dots, T$, $j = 1, \dots, n$.

$f_t(B) = +\infty$ if $|B| > b$, for $t = 0, \dots, T + 1$.

Optimal Solution Value

$f_0(\emptyset)$.

Recurrence Relation

$$f_t(B) = \min \begin{cases} w|B \cup S(t)| + f_{t+1}(B \cup S(t)), \\ w(|B \cup S(t)| - 1) + F(t + 1, v) + f_{t+1}(B \cup S(t) - \{v\}). \end{cases}$$

The main result for Algorithm M-CW1 now follows.

Theorem 2. Algorithm M-CW1 finds an optimal schedule for problem $M_r|b|C + W$ in $O(n^{b+2}b)$ time.

Proof. The first alternative in the recurrence relation computes the cost of scheduling one unit of idle time, and thus incurring the storage cost of delaying the $|B \cup S(t)|$ jobs in the buffer by one time unit. The second alternative computes the interchange cost of processing job v in M_r before any jobs which precede v in M_r^I . Since, by definition, at time $t + 1$ job v is the leftmost job in M_r^I among those in $B \cup S(t)$, no job in $B \cup S(t)$ causes an interchange cost when v is scheduled. Hence, the interchange cost created when processing v at time t equals the number of jobs that are not available before time $t + 1$ and that precede v in M_r^I , i.e. $F(t + 1, v)$. In the second alternative, exactly the jobs in $B \cup S(t) - \{v\}$ remain in the buffer at time $t + 1$. Consequently, M-CW1 compares the cost of all nondominated state transitions, and thus finds an optimal schedule.

We now consider the time complexity of M-CW1. Note that, for any given j , each job i appears in only one set $S(h)$, thus requiring $O(\max\{n, T\})$ time in the preprocessing step. Therefore, the overall time requirement for preprocessing is $O(nT)$. Also, the recurrence relation is computed for $f_t(B)$, where $0 \leq t \leq T$, and for all possible subsets B where $|B| \leq b$. This provides a total of $O(n^b T)$ applications of the recurrence relation. Since $B \cup S(t)$ can be computed in $O(n)$ time, the overall time complexity of Algorithm M-CW1 is $O(n^{b+1} T)$, which from Lemma 1 equals $O(n^{b+2} b)$ time. $\square$

We now present a second algorithm, M-CW2, for problem $M_r|b|C + W$. In order to describe Algorithm M-CW2, we need some additional notation. Let $J(i, j, k)$ be the set of jobs that are processed by the supplier in the time slots from i through j and that precede job k in M_r^I , i.e. $J(i, j, k) = \{q | q \prec^M k, i \leq \sigma^I(q) \leq j\}$. We let $J(i, j, k, l)$ be the set $J(i, j, k)$ with the addition of $l \geq 0$ idle time slots. Also, we let $|J(i, j, k)|$ denote the number of jobs in $J(i, j, k)$, and we note that $|J(i, j, k, l)| = |J(i, j, k)| + l$.

Lemma 9. If $i \preceq^S j$ and $i \prec^M j$, then in every optimal schedule M_r^* , i precedes j .

Proof. Since $i \preceq^S j$, whenever j is available for processing in M_r^* , either i has already been processed or it is also available. In the first case, the proof is complete. In the second case, it follows immediately from Lemma 8 that without loss of generality job i can be scheduled before job j in M_r^* . $\square$

Given a feasible manufacturer's schedule M_r , a set $J(i, j, k, l)$ is *compact* in M_r if all the elements of $J(i, j, k, l)$ (including the idle time slots) are sequenced consecutively in M_r , i.e. they are scheduled in time slots

$i + 1, \dots, i + |J(i, j, k, l)|$. In other words, if $J(i, j, k, l)$ is compact, then no job or idle time slot that is not in $J(i, j, k, l)$ can be scheduled in the time interval $[i + 1, i + |J(i, j, k, l)|]$.

A set $J(i, j, k, l)$ is *compact-feasible* if there exists a feasible schedule M_r such that $J(i, j, k, l)$ is compact in M_r . For a set to be compact-feasible, l can only assume certain values. There must be a sufficient number of idle time slots in M_r to account for the time slots in which no job from $J(i, j, k)$ can be scheduled. More precisely, if $J(i, j, k, l)$ is compact in M_r , in the $t - i + 1$ time slots from $i + 1$ through $t + 1$ (where $i \leq t \leq j$), only jobs in $J(i, t, k)$ can be scheduled, and the other time slots must be idle. Hence, a lower bound on l is given by:

$$l_{\min}(i, j, k) = \max \left\{ 0, \max_{t=i, \dots, j} \{t - i + 1 - |J(i, t, k)|\} \right\}. \quad (2)$$

Let h be the rightmost job of $J(i, j, k)$ in M_r^l , and let

$$P(i, j, k) = \{t | \sigma^l(h) \leq t \leq j, S^l(t) \cap (J(i, j, k) - \{h\}) = \emptyset\}.$$

Note that $P(i, j, k)$ contains all the time slots, from $\sigma^l(h)$ through j , in which no job in $J(i, j, k) - \{h\}$ is processed by the supplier. In particular, $P(i, j, k)$ contains time slot $\sigma^l(h)$ if and only if no other job in $J(i, j, k)$, other than h , becomes available from the supplier in time slot $\sigma^l(h)$. Moreover, notice that from the definition of $l_{\min}(i, j, k)$ in (2) it follows that $l_{\min}(i, j, k) = 0$ if $j < i$ and, for all $j \geq i$,

$$l_{\min}(i, j, k) = \max\{l_{\min}(i, j - 1, k), j - i + 1 - |J(i, j, k)|\}. \quad (3)$$

The following lemma identifies the time slots that are candidates to process job h .

Lemma 10. *Given an optimal schedule M_r^* , and a set $J(i, j, k, l)$, let h be the last job of $J(i, j, k, l)$ in M_r^l . Then*

- (i) *All jobs $u \in J(i, j, k) \setminus \{h\}$ such that $\sigma^l(u) \leq \sigma^l(h)$ precede h in M_r^* .*
- (ii) *Either h is scheduled in time slot $p + 1$, for some $p \in P(i, j, k)$, or h is the last job from $J(i, j, k)$ in M_r^* .*

Proof. We consider each part in turn.

- (i) The result follows immediately from Lemma 9.
- (ii) Suppose that h is not the job scheduled last among those of $J(i, j, k)$ in M_r^* , and further suppose for purposes of contradiction that h is scheduled in time slot $q + 1$ in M_r^* , where $q \notin P(i, j, k)$. Then, by definition of h and $P(i, j, k)$, $S^l(q) \cup \beta(q)$ contains a job $v \in J(i, j, k)$ such that $v \prec^M h$. Since jobs v and h are both available for processing by the manufacturer at time $q + 1$, processing job h violates Lemma 8, a contradiction. $\square$

The following two results consider the two cases in part (ii) of Lemma 10, respectively. Each result describes a decomposition of the schedule of the jobs and idle time slots of $J(i, j, k, l)$, thereby specifying which jobs precede and which jobs follow job h .

Lemma 11. *Given an optimal schedule M_r^* , and a set $J(i, j, k, l)$ that is compact in M_r^* , let h be the last job in the M_r^l schedule in that set. If h is scheduled in a time slot $p + 1$, where $p \in P(i, j, k)$, then, letting $l' = l_{\min}(i, p - 1, h)$:*

- (i) *the jobs and the idle time slots of set $J(i, j, k, l)$ that precede h in M_r^* are $A(p) = J(i, p - 1, h, l')$, where $l' \leq l$.*
- (ii) *the jobs and the idle time slots of set $J(i, j, k, l)$ that follow h in M_r^* are $B(p) = J(p + 1, j, h, l - l')$, where $l - l' \geq l_{\min}(p + 1, j, h)$.*
- (iii) *Sets $A(p)$ and $B(p)$ are compact in M_r^* .*

Proof. We consider each part in turn.

- (i) It follows from Lemma 8 that any job $q \in J(i, p - 1, h, l')$ precedes h in M_r^* . Since $J(i, j, k, l)$ is compact in M_r^* , from time slot $i + 1$ through time slot p exactly $p - i$ jobs and idle time slots are scheduled in M_r^* . Since M_r^* is feasible, $l' \leq l$.

- (ii) After h in M_r^* , the remaining jobs from $J(i, j, k, l)$ must be scheduled with $l - l'$ idle time slots, thus yielding the set $B(p) = J(p + 1, j, h, l - l')$. Since M_r^* is feasible, $l - l' \geq l_{\min}(p + 1, j, h)$.
- (iii) Note that $A(p) \cup B(p) \cup \{h\} = J(i, j, k, l)$. Then, the compactness of $A(p)$ and $B(p)$ follows from that of $J(i, j, k, l)$. $\square$

As a consequence of Lemma 11, the schedule of $J(i, j, k, l)$ consists of the jobs and idle time slots of $A(p)$, followed by job h , followed by the jobs and idle time slots of $B(p)$. In the following discussion, we refer to the case considered in Lemma 11 as *h-intermediate*.

Lemma 12. Given an optimal schedule M_r^* , and a set $J(i, j, k, l)$ that is compact in M_r^* , let h be the last job in the M_r^I schedule in that set. If h is the last job from $J(i, j, k, l)$ in M_r^* , then the jobs and idle time slots of set $J(i, j, k, l)$ that precede h in M_r^* are the set $J(i, j, h, l')$, for some $l_{\min}(i, j, h) \leq l' \leq l$, where $J(i, j, h, l')$ is compact in M_r^* .

Proof. From the hypothesis, the set of jobs of $J(i, j, k)$ preceding h in M_r^* is $J(i, j, k) - \{h\} = J(i, j, h)$. Since $J(i, j, k, l)$ is compact in M_r^* , there is a minimum number of idle time slots $l_{\min}(i, j, h)$ that must precede h in M_r^* . The remaining $l - l_{\min}(i, j, h)$ idle time slots can be scheduled either before h , thus contributing to l' , or after h . Finally, the compactness of $J(i, j, h, l')$ follows from that of $J(i, j, k, l)$. $\square$

As a consequence of Lemma 12, the schedule of $J(i, j, k, l)$ consists of the jobs of $J(i, j, k) - \{h\}$ and l' idle time slots, followed by job h , followed by $l - l'$ idle time slots. In the following discussion, we refer to the case considered in Lemma 12 as *h-last*. Note that, if $l_{\min}(i, j, h) > l$, then h cannot be scheduled last among jobs in $J(i, j, k)$ in M_r^* . Moreover in that case, from the definition of $l_{\min}(i, j, h)$, $J(i, j, h, l)$ is not compact-feasible.

Lemmas 10, 11 and 12 identify all the possible candidate time slots for job h in M_r^* , and suggest a dynamic programming algorithm to solve problem $M_r|b|C + W$. In order to construct such an algorithm, we need to identify the contribution of $J(i, j, k, l)$ to the objective function in the cases considered in Lemmas 11 and 12. Let $V(J(i, j, k, l))$ be the minimum total interchange plus storage cost to schedule the set $J(i, j, k, l)$. If we add a dummy job 0 at the end of the M_r^I schedule, then $J(1, T, 0)$ is the entire set of jobs to be scheduled. Then, $l_{\min}(1, T, 0)$ is the minimum number of idle time slots that must be inserted in any feasible schedule M_r of the r th manufacturer. Adding more than b idle time slots to $l_{\min}(1, T, 0)$, within the interval $[1, z]$, would necessarily result in buffer overflow. Therefore, the value of the optimal solution can be computed as $V(J(1, T, 0, b + l_{\min}(1, T, 0)))$.

We now describe our dynamic programming algorithm for problem $M_r|b|C + W$. We use the convention that the value obtained by minimization over an empty set is $+\infty$.

Algorithm M-CW2.

Preprocessing

$l_{\min}(i, j, k) = \max\{0, \max_{t=i, \dots, j} \{t - i + 1 - |J(i, t, k)|\}\}$, for $i, k = 1, \dots, n, j > i$.

$P(i, j, k) = \{t | \sigma^I(h) \leq t \leq j, S^I(t) \cap (J(i, j, k) - \{h\}) = \emptyset\}$, for $i, k = 1, \dots, n, j > i$.

Value Function

$V(J(i, j, k, l))$ = minimum total interchange plus storage cost to schedule the set $J(i, j, k, l)$.

Boundary Condition

$V(J(i, j, k, l)) = 0$ if $|J(i, j, k)| \leq 1, l \geq 0$.

Optimal Cost

$V(J(1, T, 0, b + l_{\min}(1, T, 0)))$.

Recurrence Relation

$$V(J(i, j, k, l)) = \min \left\{ \min_{p \in P(i, j, k)} \left\{ \begin{array}{l} V(A(p)) + V(B(p)) + w(p - \sigma^I(h)) \\ + |\{f, g | f \in A(p), g \in B(p), g <^M f\}| \\ + |J(p + 1, j, h)| \end{array} \right\}, \min_{l_{\min}(i, j, h) \leq l' \leq l} \{ V(J(i, j, h, l')) + w(i + |J(i, j, h, l')| - \sigma^I(h)) \} \right\},$$

where

$$A(p) = J(i, p-1, h, l_{\min}(i, p-1, h)),$$

$$B(p) = J(p+1, j, h, l - l_{\min}(i, p-1, h)),$$

and

h is the last job of $J(i, j, k, l)$ in M_r^I .

We now provide a brief explanation of Algorithm M-CW2, followed by a numerical example. The two alternatives in the recurrence relation represent the h -intermediate and h -last cases, respectively. Consider first the h -intermediate case. From Lemma 11, if h is scheduled in time slot $p+1$, then the schedule decomposes into three parts, namely $A(p)$, h and $B(p)$. Besides $V(A(p))$ and $V(B(p))$, we need to account for the contributions of h to storage and interchange costs, plus the interchange costs between $A(p)$ and $B(p)$. Since job h is first available for processing by the manufacturer in time slot $\sigma^I(h) + 1$ and is scheduled in time slot $p+1$, the storage cost of job h is $w(p - \sigma^I(h))$. By definition of h , there are no interchanges between h and any job in $A(p)$, whereas there is one interchange between h and every job in $B(p)$. This provides $|J(p+1, j, h)|$ interchanges. Finally, there is one interchange for each pair (f, g) such that $f \in A(p)$, $g \in B(p)$ and $g \prec^M f$.

Consider now the h -last case. From Lemma 12, and since $f \prec^M h$ for all $f \in J(i, j, h)$, we need only add to $V(J(i, j, h, l'))$ the contribution of job h to the storage costs. All jobs and idle time slots in $J(i, j, h, l')$ are scheduled before h , starting from time slot $i+1$. Hence, job h is scheduled in time slot $i+1 + |J(i, j, h, l')|$, and its contribution to storage costs is therefore $w(i + |J(i, j, h, l')| - \sigma^I(h))$.

Regarding the boundary condition, when $J(i, j, k)$ is either empty or a singleton there are no interchanges within that set. Moreover, if $|J(i, j, k)| = 1$, then there is no reason not to schedule the single job h immediately, i.e. in time slot $\sigma^I(h) + 1$. Note that this is always possible, due to the fact that $J(i, j, k, l)$ is compact. We now provide an example of Algorithm M-CW2.

Example 2. Consider the following instance of problem $M_r|b|C + W$. There are $n = 5$ jobs. The ideal supplier's schedule releases exactly one job in each time slot, i.e., $S^I = 1, 2, 3, 4, 5, 0$. From (2), $l_{\min}(1, 5, 0) = 0$. There is a buffer of capacity $b = 1$. The manufacturer's ideal schedule is $M_r^I = 2, 3, 5, 1, 4, 0$. Finally, the storage cost is $w = 1/3$.

Let $*$ denote an idle time slot. Our objective is to compute $V(J(1, n, 0, b + l_{\min}(1, 5, 0))) = V(J(1, 5, 0, 1))$.

$$V(J(1, 5, 0, 1))$$

The set $J(1, 5, 0, 1)$ includes all the jobs, so the rightmost element in M_r^I within the set is $h = 4$. We first consider the h -intermediate case. Let us consider $P(i, j, k) = P(1, 5, 0)$. Since $\sigma^I(h) = \sigma^I(4) = \{4\}$, we have $J(i, j, k) - \{h\} = J(1, 5, 0) - \{4\} = \{1, 2, 3, 5\}$ and $P(1, 5, 0) = \{t | 4 \leq t \leq 5, S^I(t) \cap (J(1, 5, 0) - \{4\}) = \emptyset\}$. The only value of t for which $S^I(t) \cap (J(i, j, k) - \{h\}) = \emptyset$ is thus $t = 4$. Therefore, we evaluate the recurrence relation for $p = 4$ only. To consider the h -intermediate case, we need to compute $l_{\min}$. For $t = 1$, $J(1, 1, 4)$ includes the jobs released from 1 to 1 which precede 4 in M_r^I , i.e. job 1 only. Thus, $|J(1, 1, 4)| = 1$ and $t - i + 1 - |J(i, t, k)| = 1 - 1 + 1 - 1 = 0$. For $t = 2$, $J(1, 2, 4) = \{1, 2\}$ and hence $t - i + 1 - |J(i, t, k)| = 2 - 1 + 1 - 2 = 0$. Similarly, for $t = 3$, $J(1, 3, 4) = \{1, 2, 3\}$, so $t - i + 1 - |J(i, t, k)| = 3 - 1 + 1 - 3 = 0$, and therefore $l_{\min}(1, 3, 4) = 0$. Thus, we have $A(p) = A(4) = J(i, p-1, h, l_{\min}(i, p-1, h)) = J(1, 3, 4, 0) = \{1, 2, 3\}$ and $B(p) = B(4) = J(p+1, j, h, l - l_{\min}(i, p-1, h)) = J(5, 5, 4, 1) = \{5, *\}$.

We now consider the h -last case. We first compute $l_{\min}(1, 5, 4)$. Besides the previously considered sets $J(1, 1, 4)$, $J(1, 2, 4)$ and $J(1, 3, 4)$, we must consider $J(1, 4, 4) = \{1, 2, 3\}$ and $J(1, 5, 4) = \{1, 2, 3, 5\}$. For $t = 4$ we have $t - i + 1 - |J(i, t, k)| = 4 - 1 + 1 - 3 = 1$, and for $t = 5$ we have $t - i + 1 - |J(i, t, k)| = 5 - 1 + 1 - 4 = 1$. Therefore, $l_{\min}(1, 5, 4) = 1$. Since $l = 1$, we have $l' = 1$. In this case $|J(i, j, h, l')| = |J(1, 5, 4, 1)| = 5$ and $\sigma^I(h) = 4$, hence the second term in the recurrence relation is $V(J(1, 5, 4, 1)) + (1 + 5 - 4)w$. Thus,

$$V(J(1, 5, 0, 1)) = \min \left\{ \begin{array}{l} V(J(1, 3, 4, 0)) + V(J(5, 5, 4, 1)) + w(4 - 4) + 1 + 1, \\ V(J(1, 5, 4, 1)) + 2w. \end{array} \right.$$

In computing $V(J(1, 3, 4, 0))$, since $l = 0$ we are forced to schedule the jobs in $J(1, 3, 4) = \{1, 2, 3\}$ in the same order as in S^l , which results in two interchanges with respect to M_r^l and no storage costs. Hence, $V(J(1, 3, 4, 0)) = 2$. From the boundary condition, $V(J(5, 5, 4, 1)) = 0$. Therefore, it only remains to compute $V(J(1, 5, 4, 1))$.

$$V(J(1, 5, 4, 1))$$

As above, we compute $h = 1$. For the h -intermediate case, we obtain $P(1, 5, 4) = \{1, 4\}$. Therefore, we evaluate the recurrence relation for $p = 1$ and $p = 4$.

$p = 1$:

$$l_{\min}(1, 0, 1) = 0,$$

$$A(1) = J(1, 0, 1, 0) = \emptyset,$$

$$B(1) = J(2, 5, 1, 1) = \{2, 3, 5, *\},$$

$p = 4$:

$$l_{\min}(1, 3, 1) = 1,$$

$$A(4) = J(1, 3, 1, 1) = \{2, 3, *\},$$

$$B(4) = J(5, 5, 1, 0) = \{5\}.$$

We now consider the h -last case. Since $l_{\min}(1, 5, 1) = 2 > 1 = l$, there are no feasible values for l' , i.e., job 1 cannot be scheduled last in $J(1, 5, 4, 1)$. Thus,

$$V(J(1, 5, 4, 1)) = \min \left\{ \begin{array}{l} \min \left\{ \begin{array}{l} V(J(1, 0, 1, 0)) + V(J(2, 5, 1, 1)) + w(1 - 1) + 0 + 3, \\ V(J(1, 3, 1, 1)) + V(J(5, 5, 1, 0)) + w(4 - 1) + 0 + 1, \end{array} \right. \\ +\infty. \end{array} \right.$$

Observe that $V(J(1, 0, 1, 0)) = 0$ and $V(J(5, 5, 1, 0)) = 0$ from the boundary condition. Therefore, to evaluate $V(J(1, 5, 4, 1))$, we need only compute $V(J(2, 5, 1, 1))$ and $V(J(1, 3, 1, 1))$.

$$V(J(2, 5, 1, 1))$$

As above, we compute $h = 5$ and $P(2, 5, 1) = \{5\}$. Therefore, we evaluate the recurrence relation for $p = 5$ only.

$p = 5$:

$$l_{\min}(2, 4, 5) = 1,$$

$$A(5) = J(2, 4, 5, 1) = \{2, 3, *\},$$

$$B(5) = J(6, 5, 5, 0) = \emptyset.$$

Since $l_{\min}(2, 5, 5) = 2 > 1 = l$, there are no feasible values for l' in the h -last case. Thus,

$$V(J(2, 5, 1, 1)) = \min \left\{ \begin{array}{l} V(J(2, 4, 5, 1)) + V(J(6, 5, 5, 0)) + w(5 - 5) + 0 + 0, \\ +\infty. \end{array} \right.$$

Observe that $V(J(6, 5, 5, 0)) = 0$ from the boundary condition. Therefore, to evaluate $V(J(2, 5, 1, 1))$, we need only compute $V(J(2, 4, 5, 1))$.

$$V(J(1, 3, 1, 1))$$

As above, we compute $h = 3$ and $P(1, 3, 1) = \{3\}$. Therefore, we evaluate the recurrence relation for $p = 3$ only.

$$l_{\min}(1, 2, 3) = 1,$$

$$A(3) = J(1, 2, 3, 1) = \{2, *\},$$

$$B(3) = J(4, 3, 3, 0) = \emptyset.$$

Since $l_{\min}(1, 3, 3) = 2 > 1 = l$, there are no feasible values for l' in the h -last case. Thus,

$$V(J(1, 3, 1, 1)) = \min \begin{cases} V(J(1, 2, 3, 1)) + V(J(4, 3, 3, 0)) + w(3 - 3) + 0 + 0, \\ +\infty. \end{cases}$$

Observe that $V(J(1, 2, 3, 1)) = 0$ and $V(J(4, 3, 3, 0)) = 0$ from the boundary condition.

$V(J(2, 4, 5, 1))$

In this case, $h = 3$ and $P(2, 4, 5) = \{3, 4\}$. We evaluate the recurrence relation for $p = 3$ and $p = 4$.

$p = 3$:

$$l_{\min}(2, 2, 3) = 0,$$

$$A(3) = J(2, 2, 3, 0) = \{2\},$$

$$B(3) = J(4, 4, 3, 1) = \{*\},$$

$p = 4$:

$$l_{\min}(2, 3, 3) = 1,$$

$$A(4) = J(2, 3, 3, 1) = \{2, *\},$$

$$B(4) = J(5, 4, 3, 0) = \emptyset.$$

Since $l_{\min}(2, 4, 3) = 2 > 1 = l$, there are no feasible values for l' in the h -last case. Thus,

$$V(J(2, 4, 5, 1)) = \min \begin{cases} \min \begin{cases} V(J(2, 2, 3, 0)) + V(J(4, 4, 3, 1)) + w(3 - 3) + 0 + 0, \\ V(J(2, 3, 3, 1)) + V(J(5, 4, 3, 0)) + w(4 - 3) + 0 + 0, \end{cases} \\ +\infty. \end{cases}$$

Observe that $V(J(2, 3, 3, 0))$, $V(J(4, 4, 3, 1))$, $V(J(2, 3, 3, 1))$ and $V(J(5, 4, 3, 0))$ are all equal to 0 from the boundary condition.

Optimal schedule construction: First, we evaluate all the above value functions.

$$V(J(2, 4, 5, 1)) = \min\{0 + 0 + w(0) + 0 + 0, 0 + 0 + w(1) + 0 + 0, +\infty\} = 0 \quad \text{for } p = 3,$$

$$V(J(1, 3, 1, 1)) = \min\{0 + 0 + w(0) + 0 + 0, +\infty\} = 0 \quad \text{for } p = 3,$$

$$V(J(2, 5, 1, 1)) = \min\{0 + 0 + w(0) + 0 + 0, +\infty\} = 0 \quad \text{for } p = 5,$$

$$V(J(1, 5, 4, 1)) = \min\{0 + 0 + w(0) + 0 + 3, 0 + 0 + w(3) + 0 + 1, +\infty\} = 2$$

for $p = 4$, since $w = 1/3$,

$$V(J(1, 5, 0, 1)) = \min\{2 + 0 + w(0) + 1 + 1, 2 + 2w\} = 8/3 \quad \text{for } l' = 1.$$

We can now backtrack to find the optimal schedule. In the summary table of the backtracking process which follows, $\{ \dots \}$ is used to denote an unordered subset of the jobs and idle times.

$V(J(i, j, k, l))$	h	p	l'	Schedule
$V(J(1, 5, 0, 1))$	4		1	{1,2,3,5,*}4
$V(J(1, 5, 4, 1))$	1	4		{2,3,*}1 {5} 4
$V(J(1, 3, 1, 1))$	3	3		{2,*}3 1 5 4
$V(J(1, 2, 3, 1))$	2	2		{*}2 3 1 5 4

The optimal schedule is * 2 3 1 5 4. The buffer contains job 1 for three time slots and job 4 for two time slots, yielding a buffer cost of $5/3$. There is one interchange between M_r^* and M_r^I , for jobs 1 and 5. Therefore, the total cost is $V(J(1, 5, 0, 1)) = \frac{8}{3}$.

The main result for Algorithm M-CW2 now follows.

Theorem 3. *Algorithm M-CW2 finds an optimal schedule for problem $M_r|b|C + W$ in $O(n^5b^4)$ time.*

Proof. First, we prove the optimality of Algorithm M-CW2. Since any optimal schedule contains only jobs and idle times, the set $J(1, T, 0, b + l_{\min}(1, T, 0))$ is compact. Lemma 10 shows that, for a compact set of jobs and idle times, there exists an optimal schedule M_r^* in which the job that is last in M_r^I is scheduled in one of two alternative ways. Lemma 11 (respectively, Lemma 12) provides a decomposition of the schedule in the first (resp., second) case. Algorithm M-CW2 compares the costs of all possible decompositions in both cases. Since, from Lemmas 11 and 12, each subset resulting from the decomposition of a compact set is also compact, this procedure can be applied recursively. It follows that Algorithm M-CW2 finds an optimal schedule for problem $M_r|b|C + W$.

We now consider the time complexity of Algorithm M-CW2. First, let us consider the sets $J(i, j, k)$. For each pair i, j , we can identify all jobs preceding k in M^I which are released by the supplier between time slots i and j , in $O(n)$ time. It follows that, since i, j and k may assume T, T and n different values respectively, all sets $J(i, j, k)$ can be generated in $O(T^2n^2)$ time. From Eq. (3), each value of $l_{\min}(i, j, h)$ can be computed in constant time. Each set $P(i, j, k)$ can be computed in $O(T)$ time by checking whether each job released by the supplier from i to j belongs to $J(i, j, k)$ or not. Hence, the whole preprocessing step can be performed in $O(T^3n)$ time. The recurrence relation requires the computation of $O(T^2nb)$ values of $V(J(i, j, k, l))$. For each $p \in P(i, j, k)$, the total number of interchanges between $A(p)$ and $B(p)$ can be computed in $O(n)$ time, as follows. We scan M^I from left to right, labelling each job $g \in B(p)$ with the number of jobs from $A(p)$ following g (i.e., still to be encountered). Adding up all the labels gives the total number of interchanges for that value of p . For the computation of $V(J(i, j, k, l))$, p and l' may assume $O(T)$ and $O(b)$ values respectively, where $O(b) < O(T)$ from Lemma 1. Thus, each value of $V(J(i, j, k, l))$ requires $O(nT)$ computation time. Therefore, the overall time complexity of the recurrence step is $O(T^3n^2b)$, which dominates that of the preprocessing step. Recalling from Lemma 1 that $T = O(nb)$, the result follows. $\square$

Since Algorithm M-CW1 has time complexity $O(n^{b+2}b)$ and Algorithm M-CW2 has time complexity $O(n^5b^4)$, neither algorithm is more efficient than the other in general.

4. Supplier's problems

In this section we address problems in which the decision maker is the supplier. Here, the jobs must be delivered by the supplier according to the G ideal manufacturers' schedules. The supplier needs to find a schedule that enables it to do so at minimum cost. Note that the supplier's problem and the manufacturer's problem are not symmetric, because of the different processing capacities of the supplier and of each manufacturer. A symmetric scenario would involve G suppliers and one manufacturer who can process G jobs during a time slot. In this case, it would be possible to derive results that are symmetric to those illustrated in this section. However, the case of one supplier and G manufacturers is common in practice, since for a given item type, each manufacturer often has one supplier. This case is therefore considered here.

Recall, from Section 2, that M_r^I and S^I denote the ideal r th manufacturer's schedule and the ideal supplier's schedule respectively, and $M^I(t)$ denotes the set of jobs requested by all manufacturers in time slot t . Let $d_t = |M^I(t)|$ and, given a supplier's schedule S , let I_t and x_t indicate the number of jobs in the buffer and the number of jobs produced

by the supplier during time slot t respectively. The vector $X = [x_1, \dots, x_T]$ is referred to as the *profile* of a supplier's schedule. Note that the following inventory balance equations hold in any feasible schedule:

$$I_t + x_t = d_{t+1} + I_{t+1}, \quad t = 1, \dots, T-1. \quad (4)$$

4.1. Interchange costs

We first consider problem $S|b|C$, where there are no storage costs, i.e. the objective is the minimization of interchange costs relative to S^I . The demand pattern of each manufacturer requires the production of at most one unit in every time slot. By assumption, the supplier can supply up to G jobs in each time slot. Jobs that have been produced by the supplier, but not yet delivered to the manufacturers, enter the buffer of capacity b .

We propose an algorithm based on the following two preliminary results. The first result is the supplier's counterpart of Lemma 7. In a supplier's problem, given a set of jobs $\mathcal{J}$, we say that $A \subseteq \mathcal{J}$ is the *rightmost* job set in S^I if no job in $\mathcal{J} \setminus A$ strictly follows a job of A in the supplier's ideal schedule S^I .

Lemma 13. *In problem $S|b|C$, given a profile, a schedule $\tilde{S}$ has the minimum number of interchanges among all the schedules having that profile, if and only if, whenever a set of jobs is scheduled in time slot $t-1$, those jobs are the rightmost in S^I among the jobs in $\beta(t) \cup M(t)$.*

Proof. (Only if.) Consider jobs i and j such that $i \prec^S j$, and both belong to $\beta(t) \cup M(t)$ for some time slot t . For purposes of contradiction, suppose that job j is scheduled strictly before job i in a schedule $\tilde{S}$ having minimum interchange cost among those having the same profile, and let S' be the supplier's schedule obtained by swapping i and j . Clearly, S' has the same profile as $\tilde{S}$. Since i and j are both in the buffer after i is scheduled in $\tilde{S}$, schedule S' is also feasible. Let ϕ be the set of jobs scheduled between $\sigma(j)$ and $\sigma(i)$ in $\tilde{S}$. The difference between the interchange costs in $\tilde{S}$ and S' is due only to i , j and the jobs in ϕ . In $\tilde{S}$, these interchange costs are $|\{k : k \in \phi, k \prec^S j\}| + |\{k : k \in \phi, i \prec^S k\}| + 1$, where the last term is due to the interchange between i and j . In S' , the interchange costs are $|\{k : k \in \phi, k \prec^S i\}| + |\{k : k \in \phi, j \prec^S k\}|$. Since $i \prec^S j$, we have $|\{k : k \in \phi, k \prec^S i\}| \leq |\{k : k \in \phi, k \prec^S j\}|$ and $|\{k : k \in \phi, j \prec^S k\}| \leq |\{k : k \in \phi, i \prec^S k\}|$. Therefore, S' has a smaller interchange cost than $\tilde{S}$, a contradiction. Thus, without loss of generality, one of the rightmost jobs in S^I can be scheduled.

(If.) The result follows from the observation that, for a given profile, all the schedules satisfying the necessary conditions have the same cost. In fact, if several choices are available for the rightmost jobs, they differ only between jobs occupying the same time slot in S^I . $\square$

We assume that the first request for a job by the manufacturers occurs in time slot 2. Also, we allow $I_1 > 0$, which is equivalent to assuming that the supplier can start producing before time slot 1. We define a supplier's profile to be packed if there exists a time slot $\bar{t}$ such that

- (i) $I_t = b$, for all $t = 1, \dots, \bar{t} - 1$,
- (ii) $I_{\bar{t}} < b$, and
- (iii) $x_t = 0$, for all $t = \bar{t}, \dots, T$.

We note that, if $\bar{t}$ exists, then (i)–(iii) uniquely define it.

Lemma 14. *There exists an optimal schedule for problem $S|b|C$ that has a packed profile.*

Proof. Let S^* be any optimal schedule, and let $\bar{t}$ be such that, for all $1 \leq t \leq \bar{t} - 1$, $I_t = b$, and $I_{\bar{t}} < b$. Note that, since it follows that $I_{\bar{t}} + d_{\bar{t}} < G + b$, Eq. (4) implies $x_{\bar{t}-1} < G$. If $x_t = 0$ from $\bar{t}$ on, then the proof is complete. Else, let $\hat{t} \geq \bar{t}$ be the first time slot in S^* after $\bar{t}$ in which some job is produced by the supplier. Since no job is produced from $\bar{t}$ through $\hat{t} - 1$, the buffer is never full in this time interval. Let j be the leftmost in S^I among the jobs produced in $\hat{t}$. Consider the schedule $\hat{S}$ obtained from S^* by scheduling job j earlier, in time slot $\bar{t} - 1$. Since, in S^* , $x_{\bar{t}-1} < G$, and the buffer is not full from $\bar{t}$ through $\hat{t} - 1$, $\hat{S}$ is also feasible. Since j is the leftmost in S^I among the jobs produced in $\hat{t}$, the number of interchanges in $\hat{S}$ is not greater than in S^* , thus $\hat{S}$ is also optimal. By repeatedly applying the above argument, we can move other jobs earlier until we obtain an optimal schedule that has a packed profile. $\square$

Lemmas 13 and 14 suggest an algorithm for finding an optimal schedule. The algorithm constructs the supplier's schedule in reverse order. In each time slot, Lemma 14 determines how many jobs must be produced by the supplier in the current time slot, while Lemma 13 indicates which jobs those should be.

Algorithm S-C.

Step 0: Given S^I , M_r^I for all $r = 1, \dots, G$, and b . Set $\beta(T) = \emptyset$, $I_{T+1} = 0$ and $d_{T+1} = 0$.

Step 1: For $t = T, \dots, 1$, do:

 Compute $x_t = \max\{0, I_{t+1} + d_{t+1} - b\}$.

 Schedule in time slot t the set $S(t)$ formed by the x_t rightmost jobs in S^I from the set $\beta(t+1) \cup M^I(t+1)$.

 Set $\beta(t) = (\beta(t+1) \cup M^I(t+1)) \setminus S(t)$, and $I_t = |\beta(t)|$.

Step 2: Schedule the job set $\beta(1)$ before time slot 1, and terminate.

Theorem 4. Algorithm S-C finds an optimal schedule for Problem $S|b|C$ in $O(nb \log(G+b))$ time.

Proof. Lemma 14 guarantees that there exists an optimal schedule which has a packed profile. Scheduling, at each time slot t , $\max\{0, I_{t+1} + d_{t+1} - b\}$ jobs guarantees that the profile of the schedule built by Algorithm S-C is packed. Since at each time slot t the scheduled jobs are selected according to Lemma 13, the schedule produced by Algorithm S-C has lowest cost among the schedules that have a packed profile, and therefore is optimal.

We consider the time complexity of Algorithm S-C. Step 1 is repeated $O(T)$ times. During the algorithm, each job is inserted into and deleted from an ordered list containing at most $G+b$ elements exactly once. The time required for each such insertion or deletion is $O(\log(G+b))$. Therefore, the overall time complexity of Algorithm S-C is $O(T \log(G+b))$, which from Lemma 1 equals $O(nb \log(G+b))$. $\square$

4.2. Interchange and storage costs

We now consider the more general supplier's problem with both interchange and storage costs, $S|b|C + W$. We propose an algorithm, S-CW, which is the supplier's counterpart of Algorithm M-CW1. The main difference from M-CW1 is that in S-CW we must decide how many jobs up to G to process in each time slot. Algorithm S-CW proceeds forward in time. The set of jobs to be processed in time slot t is determined by the jobs which are in the buffer during time slot $t+1$ and by schedule M^I .

Let $f_t(B)$ denote the minimum cost for the problem restricted to time slots from 0 through t , given that the set of jobs in the buffer during time slot $t+1$ is $\beta(t+1) = B$. We observe that Lemma 13, which is proved for problem $S|b|C$, also holds for problem $S|b|C + W$. This is because the storage cost is determined only by the profile, and not by which jobs are scheduled. Thus from Lemma 13, if the supplier decides to produce x_t jobs in time slot t , these are the x_t rightmost jobs in S^I from the set $B \cup M^I(t+1)$. We denote this set by $V(x_t)$. We also denote by $F(t, h)$ the number of interchanges due to the supplier processing job h in time slot t . This equals the number of jobs which have already been processed by the supplier, but which follow h in S^I . Since $h \in V(x_t)$, all the jobs in $\beta(t)$ precede h in S^I and therefore do not contribute to $F(t, h)$, thus $F(t, h) = |\{i | i \in \bigcup_{j=1}^t M^I(j), h \prec^S i\}|$.

There are three cost components in $f_t(B)$. The first is the storage cost due to the jobs in the buffer during time slot $t+1$, i.e., $w|B|$. The second is the total interchange cost associated with the jobs in $V(x_t)$, which equals $\sum_{h \in V(x_t)} F(t, h)$. The last component is the optimal cost up to time slot $t-1$, with $\beta(t)$ including all the jobs of $B \cup M^I(t+1)$ which are not in $V(x_t)$. This discussion suggests the following algorithm.

Algorithm S-CW.

Preprocessing

$F(t, h) = |\{i | i \in \bigcup_{j=1}^t M^I(j), h \prec^S i\}|$, for $t = 1, \dots, T$, $h = 1, \dots, n$.

Value Function

$f_t(B)$ = minimum cost for the problem from 0 to t , given that $\beta(t+1) = B$.

Boundary Conditions

$f_0(B) = w|B|$, for all $B \subseteq N$, $|B| \leq b$.

$f_t((B \cup M^I(t+1)) - V(x_t)) = +\infty$, if $x_t > d_{t+1} + |B|$.

$f_t((B \cup M^I(t+1)) - V(x_t)) = +\infty$, if $x_t < d_{t+1} + |B| - b$.

Optimal Solution Value

$f_T(\emptyset)$.

Recurrence Relation

$$f_t(B) = \min_{x_t=0,\dots,G} \left\{ w|B| + \sum_{h \in V(x_t)} F(t, h) + f_{t-1}((B \cup M^I(t+1)) - V(x_t)) \right\}.$$

Note that, since $0 \leq I_t \leq b$ and $I_{t+1} = |B|$, from Eq. (4), x_t must be at least $d_{t+1} + |B| - b$ and must not exceed $d_{t+1} + |B|$. The boundary conditions enforce these requirements.

Theorem 5. Algorithm S-CW finds an optimal schedule for problem $S|b|C + W$ in $O(n^{b+1}b(G+b)\log(G+b))$ time.

Proof. From Lemmas 13 and 14, Algorithm S-CW compares the cost of all nondominated feasible partial schedules, and thus finds an optimal schedule. In the preprocessing step, the computation of $F(t, h)$ can be performed in $O(Tn)$ time, using a labelling procedure similar to that in the proof of Theorem 3. There are $O(n^b)$ possible sets B for each time slot t , for $t = 0, \dots, T-1$. For each pair (t, B) , the recurrence relation requires computing a minimum over $O(G)$ values. All sets $V(x_t)$ can be obtained by ordering the set $B \cup M^I(t+1)$, which requires $O((G+b)\log(G+b))$ time. Therefore, the overall time complexity of Algorithm S-CW is $O(Tn^b(G+b)\log(G+b))$, which from Lemma 1 equals $O(n^{b+1}b(G+b)\log(G+b))$. $\square$

5. Combined problems

In this section, we analyze the combined problem $S, M|b|C$, for the special case where $G = 1$. Hence, we omit the subscript on M_r . In this situation, the supplier and the only manufacturer have their own ideal schedules, which may process the jobs in different orders. The first objective we consider is the minimization of overall interchange costs.

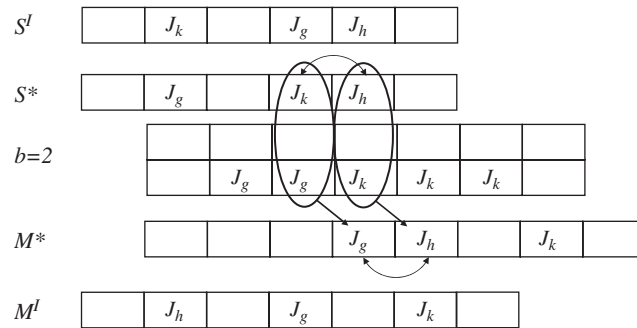
Given two schedules S and M , we say that they are b -compatible if the manufacturer can feasibly schedule the jobs according to M after the supplier releases them according to S , when the buffer has capacity b . Also, given two schedules S and M , we say that a job k is a *no-wait job* if it is processed by the supplier and by the manufacturer in consecutive time slots, i.e. if $\mu(k) = \sigma(k) + 1$; otherwise, if $\mu(k) > \sigma(k) + 1$, it is a *wait job*. Our main result now follows.

Theorem 6. Given the ideal schedules M^I and S^I , let M^* and S^* be optimal schedules for problems $M|b|C$ and $S|b|C$, respectively. We let k_m (resp., k_s) denote the minimum number of interchanges between M^I and M^* (resp., S^I and S^*).

- (i) There exist two schedules $\mathcal{S}_i^I$ and $\mathcal{M}_i^*$ which are b -compatible, such that $\mathcal{S}_i^I$ differs from S^I by i interchanges and $\mathcal{M}_i^*$ differs from M^I by $k_m - i$ interchanges, for $1 \leq i \leq k_m$.
- (ii) There exist two schedules $\mathcal{M}_j^I$ and $\mathcal{S}_j^*$ which are b -compatible, such that $\mathcal{M}_j^I$ differs from M^I by j interchanges and $\mathcal{S}_j^*$ differs from S^I by $k_s - j$ interchanges, for $1 \leq j \leq k_s$.
- (iii) There do not exist two schedules M' and S' that are b -compatible, such that, if S' differs from S^I by q_s interchanges, and M' differs from M^I by q_m interchanges, $q_s + q_m < k_m$.

Proof. If $k_m = 0$ or $k_s = 0$, then the theorem is trivially true. We consider each part in turn.

(i) We first consider $i = 1$. Since $k_m > 0$, there is at least one pair of consecutively processed jobs g, h in M^* such that $h \prec^M g$. Note that g is released by the supplier before h . From the “only if” part of Lemma 7, when g is scheduled in M^* , h is not available yet, and therefore since h immediately follows g in M^* , h is no-wait. We consider two subcases, depending on whether g is a no-wait or a wait job. Assume first that g is no-wait. Then, g and h are consecutive in both S^I and M^* . By interchanging jobs g and h in both S^I and M^* , we obtain two new schedules, $\mathcal{S}_1^I$ and $\mathcal{M}_1^*$, respectively, such that there is exactly one interchange between $\mathcal{S}_1^I$ and S^I , and exactly $k_m - 1$ interchanges between $\mathcal{M}_1^*$ and M^I . Moreover, these two schedules are b -compatible by construction. Alternatively, assume that job g is a wait job, and consider the job k processed by the supplier in time slot $\sigma^I(h) - 1$. Observe that, since jobs g and h are consecutive in M^* , job k is scheduled after them in M^* (see Fig. 3). From the “only if” part of Lemma 7, $h \prec^M k$ and $g \prec^M k$. Hence,

Fig. 3. Illustration of the proof of Theorem 6 in the case g is a wait-job.

by interchanging jobs g and h in M^* , and jobs k and h in S^I , we again obtain two schedules, $\mathcal{S}_1^I$ and $\mathcal{M}_1^*$, respectively. The only difference in buffer usage resulting from these interchanges is that the pair of schedules S^I, M^* has job k in the buffer during time slot $\sigma^I(h)$, whereas the pair $\mathcal{S}_1^I, \mathcal{M}_1^*$ has job g in the buffer during that time slot. Thus, $\mathcal{S}_1^I$ and $\mathcal{M}_1^*$ are b -compatible. Moreover, $\mathcal{S}_1^I$ differs from S^I by exactly one interchange, and $\mathcal{M}_1^*$ differs from M^I by exactly $k_m - 1$ interchanges. For $i > 1$, we set $S^I = \mathcal{S}_1^I$, $M^* = \mathcal{M}_1^*$ and repeat the above argument.

(ii) The proof is similar to that of part (i), by symmetry.

(iii) For purposes of contradiction, assume that two b -compatible schedules S' and M' exist, such that $q_s + q_m < k_m$. From part (ii), we can find two b -compatible schedules S'' and M'' requiring $q_s - 1$ and $q_m + 1$ interchanges, where the total number of interchanges required is still $(q_s - 1) + (q_m + 1) < k_m$. By applying this argument another $q_s - 1$ times, we can find two b -compatible schedules $\mathcal{S}_{q_s}^* = S^I$ and $\mathcal{M}_{q_s}^I$, such that $\mathcal{M}_{q_s}^I$ differs from M^I by exactly $q_s + q_m < k_m$ interchanges. This implies that the schedule found by Algorithm M-C which contained k_m interchanges was not optimal, a contradiction. $\square$

Theorem 6 has an important implication for the solvability of problem $S, M|b|C$.

Theorem 7. An optimal schedule for problem $S, M|b|C$ where $G = 1$ can be found in $O(nb \log b)$ time.

Proof. It follows from Theorem 6 that an optimal schedule for problem $M|b|C$ and an optimal schedule for problem $S|b|C$ are both optimal for problem $S, M|b|C$ where $G = 1$. Therefore, we can apply either Algorithm M-C which runs in $O(nb \log b)$ time (see Theorem 1), or Algorithm S-C which runs in $O(nb \log(G + b))$ time in general (see Theorem 4) and thus in $O(nb \log b)$ time where $G = 1$. $\square$

The complexity class of problem $S, M|b|C$, where $G \geq 2$, remains open.

A second implication of Theorem 6 is that, when there are no storage costs, the interchange cost incurred by the two parties can always be divided between them in various ways. However, the supplier and the manufacturer *cannot* decrease the total interchange cost by cooperation.

On the other hand, if storage costs are also considered as in problem $S, M|b|C + W$, cooperation between the supplier and manufacturer *can* provide an improvement in total cost, as shown by the following example.

Example 3. Consider the following instance of problem $S, M|b|C + W$: $b = 2$; $S^I = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10$; $M^I = 4, 5, 2, 1, 3, 9, 10, 6, 8, 7$; $w = 0.1$.

Let “*” denote an idle time period.

The optimal solution, M^* , to problem $M|2|C + W$ (which can be found by Algorithm M-CW1 or Algorithm M-CW2) is the following, with total cost of $C + W = 4 + 19(0.1) = 5.9$

S^I	1	2	3	4	5	6	7	8	9	10
M^*	*	2	*	4	5	1	3	6	9	10
M^I	4	5	2	1	3	9	10	6	8	7

The optimal solution, S^* , to problem $S|2|C + W$ can be found by Algorithm S-CW. Note that, for the manufacturer

to process job 4 in time slot 2, the supplier must process jobs 1 and 2 before time slot 1. The algorithm yields the following solution, also with total cost of $C + W = 4 + 19(0.1) = 5.9$

S^I			1	2	3	4	5	6	7	8	9	10	
S^*	1	2	4	5	3	6	7	9	10	*	8	*	
M^I				4	5	2	1	3	9	10	6	8	7

However, the following solution for the combined problem $S, M|2|C + W$ has a smaller total cost of $C + W = 4 + 16(0.1) = 5.6$

S^I			1	2	3	4	5	6	7	8	9	10					
S^*	1	2	4	5	*	*	3	6	7	8	9	10					
M^*				4	5	2	1	3	6	*	*	9	10	8	7		
M^I				4	5	2	1	3	9	10	6	8	7				

This example shows that cooperation can provide a reduction in total system cost when that cost includes storage. Finally, we note that the complexity class of problem $S, M|b|C + W$ remains open.

6. Concluding remarks

In this paper, we present various models for coordinating scheduling decisions between a supplier and several manufacturers. We provide polynomial time scheduling algorithms that enable a manufacturer to schedule jobs optimally, taking into account the supplier's delivery schedule. Other efficient scheduling algorithms enable a supplier to schedule jobs optimally, while meeting the requirements of several manufacturers. A useful property of our algorithms is that their running times increase at most polynomially with the number of manufacturers. We also derive properties of the more general situation where the supplier and the manufacturers may cooperate to reduce total cost. Here we demonstrate that, if only interchange costs are considered, then no reduction in overall system cost can result from cooperation between a supplier and a manufacturer. However, if storage costs are also considered, then such cooperation can reduce the overall system cost.

A number of important issues remain open for future investigation. The first is the resolution of the computational complexity of the two open problems, $S, M|b|C$ where $G \geq 2$ and $S, M|b|C + W$, discussed here. Second, it would be useful to extend our models to multiple stage supply chains. Third, it is important to develop mechanisms by which decision makers at different stages of a supply chain can cooperate, and incentives for this cooperation. Fourth, one might consider extending the approach to account for further issues, such as due dates for the jobs. Finally, there is a need to develop models that allow for imperfect or asymmetric information in the negotiations between suppliers and manufacturers. We hope that our work will stimulate further research to investigate how decision makers with different ideal schedules in a supply chain can most effectively cooperate.

Acknowledgments

This work is supported in part by the National Science Foundation (grant number DMI-9821033), by the Summer Fellowship Program of the Fisher College of Business (The Ohio State University), by NATO Collaborative Research Grant CRG 950773, and by the Italian Ministry of Scientific Research Grant "Models and Algorithms for Operational Planning in Distributed Systems".

References

- [1] A. Agnetis, P. Detti, C. Meloni, D. Pacciarelli, Set-up coordination between two stages of a supply chain, *Ann. Oper. Res.* 107 (2001) 15–32.
- [2] R. Anupindi, R. Akella, Diversification under supply uncertainty, *Manage. Sci.* 39 (1993) 944–963.
- [3] R.D. Banker, I.S. Khosla, Economics and operations management: a research perspective, *J. Operations Management* 12 (1995) 423–425.
- [4] D.E. Blumenfeld, L.D. Burns, C.F. Daganzo, Synchronizing production and transportation schedules, *Transportation Res. B* 25 (1991) 23–37.
- [5] P. Chandra, M.L. Fisher, Coordination of production and distribution planning, *European J. Oper. Res.* 72 (1994) 503–517.
- [6] Z.-L. Chen, N.G. Hall, Supply chain scheduling: assembly systems, Working Paper, Fisher College of Business, The Ohio State University, Columbus, OH, 2000.
- [7] M. Dawande, H.N. Geismar, N.G. Hall, C. Sriskandarajah, Supply chain scheduling: distribution systems, Working Paper, Fisher College of Business, The Ohio State University, Columbus, OH, 2004.

- [8] S.S. Erengüç, N.C. Simpson, A.J. Vakharia, Integrated production/distribution planning in supply chains: an invited review, *European J. Oper. Res.* 115 (1999) 219–236.
- [9] R.L. Graham, E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, Optimization and approximation in deterministic machine scheduling: a survey, *Ann. Discrete Math.* 5 (1979) 287–326.
- [10] N.G. Hall, C.N. Potts, Supply chain scheduling: batching and delivery, *Oper. Res.* 51 (4) (2002) 566–584.
- [11] S.R. Helper, M. Sako, Supplier relations in Japan and the United States: are they converging? *Sloan Manage. Rev.* 36 (3) (1995) 77–84.
- [12] A. Hernandez, *Just-in-Time Manufacturing: A Practical Approach*, Prentice-Hall, Englewood Cliffs, NJ, 1989.
- [13] A.P. Hurter, M.G. Van Buer, The newspaper production/distribution problem, *J. Business Logistics* 17 (1996) 85–107.
- [14] U.S. Karmarkar, Integrative research in marketing and operations management, *J. Marketing Res.* 33 (1996) 125–133.
- [15] C.-Y. Lee, Z.-L. Chen, Machine scheduling with transportation considerations, *J. Scheduling* 4 (2001) 3–24.
- [16] M. Mahajan, R. Rama, V. Sundararajan, On sorting by 3-bounded transpositions, *Electron. Notes Discrete Math.* 15 (2003) 117–120.
- [17] M. Moses, S. Seshadri, Policy mechanisms for supply chain coordination, *IIE Trans.* 32 (2000) 245–262.
- [18] J.A. Muckstadt, R.O. Roundy, *Logistics of Production and Inventory*, Handbooks in Operations Research and Management Science, vol. 4, North-Holland, Amsterdam, 1993.
- [19] D.O. Nellesmann, L.F. Smith, Just-in-time vs. just-in-case production systems borrowed back from Japan, *Production Inventory Manage.* 23 (1982) 12–21.
- [20] T. Ohno, *Toyota Production System: Beyond Large-Scale Production*, Productivity Press, Cambridge, MA, 1988.
- [21] P.S. Ow, S.F. Smith, R. Howie, A cooperative scheduling system, in: M.D. Oliff (Ed.), *Expert Systems and Intelligent Manufacturing*, Elsevier Science Publishing, Amsterdam, 1988, pp. 43–56.
- [22] A.M. Sarmiento, R. Nagi, A review of integrated analysis of production–distribution systems, *IIE Trans.* 31 (1999) 1061–1074.
- [23] D.J. Thomas, P.M. Griffin, Coordinated supply chain management, *European J. Oper. Res.* 94 (1996) 1–15.
- [24] Z.K. Weng, Pricing and ordering strategies in manufacturing and distribution alliances, *IIE Trans.* 29 (1997) 681–692.